unit_linked

November 5, 2026

1 PROTOTYPE UNIT-LINKED LIABILITIES

The purpose of this notebook is to showcase how unit-linked pension liabilities work inside the OSEM model, using the same production classes and loop helpers that `main.py` calls when `liability_mode=unit_linked`.

Unit-linked products differ from the static cash-flow liability mode: they do **not** upload a pre-scheduled outflow vector. Cash flows emerge each period from policy state (account value, premiums, fees) and stochastic mortality / lapse liquidations.

Detailed methodology: `Documentation/Unit_Linked_Methodology.md` and `Documentation/Unit_Linked_Methodology.html`.

[1]:
```
import os
import pandas as pd
import datetime as dt
from pathlib import Path
import sys

base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
```

[2]:
```
from osem.ConfigurationClass import Configuration
from osem.ImportData import (
    get_configuration,
    get_settings,
    get_unit_linked_policies,
    get_unit_linked_fund,
    get_society,
)
from osem.UnitLinkedClasses import UnitLinkedPolicy, UnitLinkedFund,␣
 ↪UnitLinkedPortfolio
from osem.SocietyClass import Society
from osem.MainLoop import (
    capitalize_policies,
    apply_premiums,
    apply_admin_fees,
```

```
    apply_mortality,
    apply_lapse,
    process_unit_linked_period,
)
```

1.1 Input files

Unit-linked runs need the usual ALM configuration plus three liability-specific files under `Input/`. This notebook reads the copies in `notebooks/data/Input`, named by `notebooks/data/notebooks.ini`, so its results do not change with the model's live `Input` folder.

1.1.1 Parameters.csv

Adds two fields used by the UL path:

- `liability_mode` … `cashflow` (default) or `unit_linked`
- `random_seed` … integer seed for reproducible mortality / lapse draws

Other parameters (modelling date, projection length, EIOPA files) are shared with the asset side.

1.1.2 Unit_Linked_Policies.csv

In-force policy file at modelling date. Columns:

- `Policy_ID` … unique identifier
- `Birth_Date` … for age-based mortality (`dd/mm/yyyy`)
- `Is_Female` … 0/1 selects mortality column
- `Is_Guaranteed` … 0/1; if 1, GV is capitalized with MV
- `Premium` … opening annual premium
- `MV` … opening account / reserve value
- `GV` … opening guaranteed value

1.1.3 Unit_Linked_Fund.csv

Single-pool fund parameters (one row in the MVP):

- `Fund_ID`
- `Lapse_Rate` … annual lapse probability
- `Admin_Fee` … annual fee as share of MV
- `Entry_Fee` … share of gross premium to insurer
- `Premium_Growth` … annual premium growth rate

1.1.4 mortality.csv

Age-indexed annual mortality rates with columns `AGE`, `MALE`, `FEMALE` (already used by `Society`).

Set up the base folder

[3]:
```
base_folder = os.getcwd()  # Get current working directory
```

Most of the run settings are saved in the configuration file, here the notebooks' copy of `ALM.ini`:

[4]:
```
conf: Configuration
conf = get_configuration(os.path.join(base_folder, "notebooks", "data",␣
↪"notebooks.ini"), os)
```

These lines of code extract the absolute locations of inputs:

[5]:
```
parameters_file = conf.input_parameters
ul_policies_file = conf.input_unit_linked_policies
ul_fund_file = conf.input_unit_linked_fund
mortality_file = conf.input_mortality
```

[6]:
```
paramfile = pd.read_csv(parameters_file).set_index("Parameter")
```

[7]:
```
display(paramfile)
```

Parameter	Value
EIOPA_param_file	notebooks/data/Input/Param_no_VA.csv
EIOPA_curves_file	notebooks/data/Input/Curves_no_VA.csv
country	Slovenia
n_proj_years	50
Precision	1E-10
Tau	0.0001
Modelling_Date	29/04/2023
liability_mode	cashflow
random_seed	42

The settings object holds modelling date, projection horizon, liability mode, and random seed.

`liability_mode` only gates which liability branch `main.py` takes. This notebook calls the unit-linked helpers directly, so it runs the same way whether its copy of `Parameters.csv` says `cashflow` or `unit_linked` (see the last section for how to switch).

[8]:
```
settings = get_settings(parameters_file)
```

[9]:
```
print("modelling_date:", settings.modelling_date)
```

```
modelling_date: 2023-04-29
```

[10]:
```
print("liability_mode:", settings.liability_mode)
```

```
liability_mode: cashflow
```

[11]:
```
print("random_seed:", settings.random_seed)
```

```
random_seed: 42
```

[12]:
```
print("end_date:", settings.end_date)
```

```
end_date: 2073-04-29
```

1.2 Loading unit-linked objects

A Python generator reads the policy CSV into `UnitLinkedPolicy` instances keyed by `policy_id`.

[13]:
```
policy_generator = get_unit_linked_policies(ul_policies_file)
ul_policies = {policy.policy_id: policy for policy in policy_generator}
```

Inspect a single policy (static metadata is frozen after import):

[14]:
```
example_id = sorted(ul_policies.keys())[0]
policy = ul_policies[example_id]
```

[15]:
```
display(ul_policies.keys())
```

```
dict_keys([10010012, 10022286, 10032029, 10043590, 10050930, 10064949, 10075652,␣
↪10085812, 10090981, 10100125])
```

[16]:
```
print(policy)
```

```
UnitLinkedPolicy(policy_id=10010012, birth_date=datetime.date(1970, 3, 15),
is_female=True, is_guaranteed=False, premium=5000.0, mv=120000.0, gv=0.0)
```

[17]:
```
print("Age at modelling date:", policy.age_at(settings.modelling_date))
```

```
Age at modelling date: 53
```

Fund parameters and mortality society:

[18]:
```
ul_fund = get_unit_linked_fund(ul_fund_file)
```

[19]:
```
display(ul_fund)
```

```
UnitLinkedFund(fund_id=1, lapse_rate=0.04, admin_fee=0.01, entry_fee=0.03,␣
↪premium_growth=0.02)
```

[20]:
```
society = get_society(mortality_file)
```

[21]:
```
print("Male q at age 50:", society.mortality_rate(50, is_female=False))
```

```
Male q at age 50: 0.020122
```

[22]:
```
print("Female q at age 50:", society.mortality_rate(50, is_female=True))
```

```
Female q at age 50: 0.009009
```

1.3 Portfolio wrapper and state DataFrames

`UnitLinkedPortfolio` mirrors the equity/bond portfolio pattern. Simulation state lives in DataFrames (rows = `policy_id`, columns = modelling dates), not on the frozen dataclasses.

There is no separate boolean "active" matrix: `init_policy_state_to_dataframe` returns just `(mv_df, gv_df, premium_df)`, and a policy that dies or lapses has its row dropped from all three

(see `apply_mortality` / `apply_lapse` in steps 4-5 below) rather than being flagged inactive in place.

[23]:
```
ul_ptf = UnitLinkedPortfolio(ul_policies)
ul_mv_df, ul_gv_df, ul_premium_df = ul_ptf.init_policy_state_to_dataframe(
    modelling_date=settings.modelling_date
)
```

[24]:
```
print("Opening UL reserve:", ul_ptf.total_reserve(ul_mv_df, settings.
  ↪modelling_date))
```

```
Opening UL reserve: 914541.0
```

[25]:
```
display(ul_mv_df)
```

	2023-04-29
10010012	120000.0
10022286	250000.0
10032029	75000.0
10043590	180000.0
10050930	95000.0
10064949	8816.0
10075652	44745.0
10085812	12006.0
10090981	36414.0
10100125	92560.0

[26]:
```
display(ul_premium_df)
```

	2023-04-29
10010012	5000.0
10022286	8000.0
10032029	3000.0
10043590	6000.0
10050930	4500.0
10064949	88.0
10075652	447.0
10085812	120.0
10090981	364.0
10100125	926.0

[27]:
```
print(f"{ul_mv_df.shape[0]} policies in force at the modelling date.")
print("There is no separate boolean active-flag matrix: a policy that later␣
  ↪dies or lapses")
print("simply has its row dropped from mv_df / gv_df / premium_df (see steps␣
  ↪4-5 below).")
```

```
10 policies in force at the modelling date.
There is no separate boolean active-flag matrix: a policy that later dies or
lapses
```

simply has its row dropped from mv_df / gv_df / premium_df (see steps 4-5 below).

1.4 Dual cash accounts

- `bank_account` — fund / policyholder pool cash (flows into `trade()` in the full ALM run)
- `company_account` — insurer fee revenue (entry fee and admin fee; **not** passed through trading)

In this notebook we track toy cash balances to illustrate the methodology without running the full asset ALM loop.

[28]:
```
previous_date = settings.modelling_date
bank_account = pd.DataFrame(data=[0.0], columns=[previous_date])
company_account = pd.DataFrame(data=[0.0], columns=[previous_date])
```

[29]:
```
display(bank_account)
```

```
   2023-04-29
0         0.0
```

[30]:
```
display(company_account)
```

```
   2023-04-29
0         0.0
```

1.5 Period length

Modelling steps need not be exactly one year. OSEM uses the same convention as the asset side:

$$\Delta t = \frac{currentDate - previousDate}{365.25}$$

Annual fund rates and mortality are converted to the elapsed period (see steps below).

[31]:
```
current_date = previous_date + dt.timedelta(days=365)
time_frac = (current_date - previous_date).days / 365.25
```

[32]:
```
print("previous_date:", previous_date)
```

```
previous_date: 2023-04-29
```

[33]:
```
print("current_date:", current_date)
```

```
current_date: 2024-04-28
```

[34]:
```
print("time_frac:", time_frac)
```

```
time_frac: 0.999315537303217
```

1.6 Where UL sits in the ALM main loop

In `main.py`, after equities/bonds are marked to market and `Portfolio return` is computed, and **before** proportional `trade()`:

1. Capitalize MV (and GV if guaranteed) by the **period** portfolio return
2. Grow premiums; credit gross premium to `bank_account`; credit entry fee to `company_account`; add net to MV
3. Deduct admin fee from MV; credit `company_account`
4. Stochastic mortality (full MV liquidation if drawn)
5. Stochastic lapse on survivors
6. UL reserve = sum of active MV

This notebook uses an illustrative period return `r_period` (in the full run it comes from the asset portfolio).

[35]:
```
# Illustrative period return from the asset MTM step in main.py
# (that value already embeds time_frac on the asset side)
r_period = 0.05
print("Illustrative portfolio_return (period factor):", r_period)
```

```
Illustrative portfolio_return (period factor): 0.05
```

1.7 Step-by-step walkthrough

Carry forward previous state into the new date column, then apply each helper used by `process_unit_linked_period`.

[36]:
```
# Working copies so cells remain re-runnable independently after reload
mv_df = ul_mv_df.copy()
gv_df = ul_gv_df.copy()
premium_df = ul_premium_df.copy()

mv_df[current_date] = mv_df[previous_date]
gv_df[current_date] = gv_df[previous_date]
premium_df[current_date] = premium_df[previous_date]
bank_account[current_date] = bank_account[previous_date]
company_account[current_date] = company_account[previous_date]
```

1.7.1 1) Capitalize

Active policies:

$$MV_t = MV_t * (1 + r_t)$$

Where: - $r_t ...$ is the calculated portfolio return at time t. - MV_t^i ... is the market value of the policy i at time t.

Guaranteed policies also capitalize GV the same way.

[37]:
```
print("MV before capitalize:")
display(mv_df[[previous_date, current_date]])

mv_df, gv_df = capitalize_policies(
    mv_df, gv_df, ul_policies, current_date, portfolio_return=r_period
)

print("MV after capitalize:")
display(mv_df[[current_date]])
print("GV after capitalize (guaranteed policies only change):")
display(gv_df[[current_date]])
```

```
MV before capitalize:
```

	2023-04-29	2024-04-28
10010012	120000.0	120000.0
10022286	250000.0	250000.0
10032029	75000.0	75000.0
10043590	180000.0	180000.0
10050930	95000.0	95000.0
10064949	8816.0	8816.0
10075652	44745.0	44745.0
10085812	12006.0	12006.0
10090981	36414.0	36414.0
10100125	92560.0	92560.0

```
MV after capitalize:
```

	2024-04-28
10010012	126000.00
10022286	262500.00
10032029	78750.00
10043590	189000.00
10050930	99750.00
10064949	9256.80
10075652	46982.25
10085812	12606.30
10090981	38234.70
10100125	97188.00

```
GV after capitalize (guaranteed policies only change):
```

	2024-04-28
10010012	0.0
10022286	210000.0
10032029	0.0
10043590	0.0
10050930	0.0
10064949	0.0
10075652	0.0

```
10085812          0.0
10090981          0.0
10100125          0.0
```

1.7.2 2) Premiums and entry fee

$$P_t^{i,G} = P_{t-1}^{i,G} * (1 + pg_t)^{\Delta t}$$

$$F_t^{i,P} = P_t^{i,G} * f_t^P$$

$$P_t^{i,N} = P_t^{i,G} - F_t^{i,P}$$

$$b_t^c = b_t^c + F_t^{i,P}$$

$$MV_t^i = MV_t^i + P_t^{i,N}$$

Where: - pg_t ... premium growth for the period t. - Δt ... is the time fraction or the length of the period (in years) - $P_t^{i,G}$ ... is the gross premium for the policy i at time t - f_t^P ... is the entry fee charged by the asset manager at time t - $F_t^{i,P}$ ... is the size of the the entry fee, collected at time t in policy i - $P_t^{i,N}$ ... is the net premium for the policy i at time t - b_t^c ... is the company bank account at time t - MV_t^i ... is the market value of policy i at time t

[38]:
```
mv_df, premium_df, gross_premium, entry_fee = apply_premiums(
    mv_df, premium_df, ul_fund, current_date, time_frac
)
bank_account[current_date] += gross_premium
company_account[current_date] += entry_fee

print("gross_premium:", gross_premium)
print("entry_fee (to company_account):", entry_fee)
print("bank_account:", float(bank_account.loc[0, current_date]))
print("company_account:", float(company_account.loc[0, current_date]))
display(mv_df[[current_date]])
display(premium_df[[current_date]])
```

```
gross_premium: 29013.50674363152
entry_fee (to company_account): 870.4052023089457
bank_account: 29013.50674363152
company_account: 870.4052023089457
```

```
            2024-04-28
10010012  130946.932948
10022286  270415.092717
10032029   81718.159769
10043590  194936.319538
```

```
10050930  104202.239653
10064949    9343.866020
10075652   47424.505806
10085812   12725.026391
10090981   38594.836719
10100125   98104.171982
```

```
            2024-04-28
10010012  5099.930874
10022286  8159.889399
10032029  3059.958525
10043590  6119.917049
10050930  4589.937787
10064949    89.758783
10075652   455.933820
10085812   122.398341
10090981   371.274968
10100125   944.507198
```

1.7.3 3) Admin fee

Actual administration fee:

$$adminFee_t = 1 - (1 - f_t^A)^{\Delta t}$$

$$F_t^{i,A} = MV_t^i * adminFee_t$$

$$MV_t^i = MV_t^i - F_t^{i,A} = MV_t^i * (1 - adminFee_t)$$

$$b_t^c = b_t^c + F_t^{i,A}$$

Where: - f^A ... is the relative size of the administration fee (on a yearly basis) - $F_t^{i,A}$ ... is the absolute size of the administration fee collected by the company - $adminFee_t$ ... is the relative size of the administration fee given the length of the period t

[39]:
```
mv_df, admin_fee = apply_admin_fees(mv_df, ul_fund, current_date, time_frac)
company_account[current_date] += admin_fee

print("admin_fee (to company_account):", admin_fee)
print("company_account:", float(company_account.loc[0, current_date]))
display(mv_df[[current_date]])
```

```
admin_fee (to company_account): 9877.38012648758
company_account: 10747.785328796526
```

```
              2024-04-28
10010012  129638.355408
10022286  267712.783401
10032029   80901.534697
10043590  192988.283920
10050930  103160.926907
10064949    9250.490994
10075652   46950.583723
10085812   12597.862788
10090981   38209.151194
10100125   97123.798382
```

1.7.4 4) Mortality sampling (stochastic)

For each **active** policy (sorted by `policy_id`):

$$mF_t^i = 1 - (1 - q_t^i)^{\Delta t}$$

$u \sim \text{Uniform}(0, 1)$

If the fund is guaranteed then:

$$b_t^f = b_t^f - max(MV_t^i, GV_t^i) I_{u<mF_t^i}$$

Otherwise:

$$b_t^f = b_t^f - MV_t^i I_{u<mF_t^i}$$

If the draw liquidates the policy it is **removed** from the state matrices entirely: `mv_df`, `gv_df` and `premium_df` all drop that row. It is not zeroed in place, so a liquidated policy cannot be charged a premium or a fee in any later period.

Where: - b_t^f ... is the fund's own bank account - mF_t^i ... is the the probability, the client in policy i dies within period t. - q_t^i ... is the probability that a person that is the same age as client in policy i in the same year as the start of period t.

Using a forced RNG that always survives makes the path deterministic for teaching.

[40]:
```
class AlwaysSurvive:
    def random(self):
        return 0.99

mv_df, gv_df, premium_df, death_total, deaths = apply_mortality(
    mv_df, gv_df, premium_df, ul_policies, society, current_date, time_frac,␣
 ↪AlwaysSurvive()
)
bank_account[current_date] -= death_total

print("deaths:", deaths, "death_total:", death_total)
```

```
print("policies remaining in force:", mv_df.shape[0])
display(mv_df[[current_date]])
```

```
deaths: 0 death_total: 0.0
policies remaining in force: 10
```

	2024-04-28
10010012	129638.355408
10022286	267712.783401
10032029	80901.534697
10043590	192988.283920
10050930	103160.926907
10064949	9250.490994
10075652	46950.583723
10085812	12597.862788
10090981	38209.151194
10100125	97123.798382

1.7.5 5) Lapse sampling (survivors only)

Same Bernoulli pattern with period-scaled `lapse_rate` on policies that remain active after mortality.

For each **active** policy (sorted by `policy_id`):

$$lF_t^i = 1 - (1 - l_t)^{\Delta t}$$

$u \sim \text{Uniform}(0, 1)$

If the fund is guaranteed then:

$$b_t^f = b_t^f - max(MV_t^i, GV_t^i) I_{u<lF_t^i}$$

Otherwise:

$$b_t^f = b_t^f - MV_t^i I_{u<lF_t^i}$$

If the draw liquidates the policy it is **removed** from the state matrices entirely: `mv_df`, `gv_df` and `premium_df` all drop that row. It is not zeroed in place, so a liquidated policy cannot be charged a premium or a fee in any later period.

Where: - b_t^f ... is the fund's own bank account - lF_t^i ... is the the probability, the client in policy i lapses within period t. - l_t ... is the probability that a person lapses in the same year as the start of period t.

[41]:
```
mv_df, gv_df, premium_df, surrender_total, lapses = apply_lapse(
    mv_df, gv_df, premium_df, ul_fund, current_date, time_frac, AlwaysSurvive()
)
bank_account[current_date] -= surrender_total
```

```
print("lapses:", lapses, "surrender_total:", surrender_total)
print("UL reserve:", ul_ptf.total_reserve(mv_df, current_date))
print("policies remaining in force:", mv_df.shape[0])
print("bank_account:", float(bank_account.loc[0, current_date]))
print("company_account:", float(company_account.loc[0, current_date]))
```

```
lapses: 0 surrender_total: 0.0
UL reserve: 978533.771414835
policies remaining in force: 10
bank_account: 29013.50674363152
company_account: 10747.785328796526
```

1.8 One-shot orchestrator (`process_unit_linked_period`)

The production helper carry-forwards state, runs steps 1–5, and returns cash-flow totals. Signs for summary reporting are applied in `main.py` (benefits logged negative).

[42]:
```
mv2, gv2, prem2 = ul_ptf.init_policy_state_to_dataframe(previous_date)

mv2, gv2, prem2, cfs, _ = process_unit_linked_period(
    current_date=current_date,
    previous_date=previous_date,
    portfolio_return=r_period,
    time=time_frac,
    mv_df=mv2,
    gv_df=gv2,
    premium_df=prem2,
    policies=ul_policies,
    fund=ul_fund,
    society=society,
    random_seed=settings.random_seed,
    proj_period=0,
)

display(pd.Series(cfs, name="cashflows"))
print("UL reserve:", ul_ptf.total_reserve(mv2, current_date))
print("policies remaining in force:", mv2.shape[0])
display(mv2[[current_date]])
```

```
return_credited     45727.050000
gross_premium       29013.506744
entry_fee             870.405202
admin_fee            9877.380126
death              267712.783401
surrender          192988.283920
deaths                  1.000000
lapses                  1.000000
in_force                8.000000
Name: cashflows, dtype: float64
```

```
UL reserve: 517832.70409431634
policies remaining in force: 8
```

```
             2024-04-28
10010012  129638.355408
10032029   80901.534697
10050930  103160.926907
10064949    9250.490994
10075652   46950.583723
10085812   12597.862788
10090981   38209.151194
10100125   97123.798382
```

1.9 Reproducibility

The same `random_seed` and `proj_period` produce the same draws. Different seeds can change liquidations.

[43]:
```
def run_seeded(seed: int):
    mv, gv, prem = ul_ptf.init_policy_state_to_dataframe(previous_date)
    _, _, _, cfs, _ = process_unit_linked_period(
        current_date=current_date,
        previous_date=previous_date,
        portfolio_return=r_period,
        time=time_frac,
        mv_df=mv,
        gv_df=gv,
        premium_df=prem,
        policies=ul_policies,
        fund=ul_fund,
        society=society,
        random_seed=seed,
        proj_period=1,
    )
    return cfs["deaths"], cfs["lapses"], cfs["in_force"]

print("seed 42:", run_seeded(42))
print("seed 42 again:", run_seeded(42))
print("seed 99:", run_seeded(99))
```

```
seed 42:
```

```
 (0.0, 0.0, 10.0)
seed 42 again: (0.0, 0.0, 10.0)
seed 99: (0.0, 0.0, 10.0)
```

1.10 Multi-period illustration

Roll several periods with a fixed illustrative return and the configuration seed. Track reserve, company account, and policies in force.

`proj_period` counts completed periods and starts at 0, matching `main.py`, so that the draws here reproduce those of a production run with the same seed.

The `Bank account` column goes deeply negative. That is an artifact of this standalone illustration: there is no asset book here, so death and surrender payments are drawn from an account that opens at zero and is never replenished by asset income or `trade()`. In a full run the same flows are netted against the portfolio. Read it as a cumulative fund cash flow, not as a balance.

[44]:
```
n_periods = 10
mv, gv, prem = ul_ptf.init_policy_state_to_dataframe(settings.modelling_date)
company = pd.DataFrame(data=[0.0], columns=[settings.modelling_date])
bank = pd.DataFrame(data=[0.0], columns=[settings.modelling_date])

history = []
prev = settings.modelling_date
for k in range(1, n_periods + 1):
    curr = prev + dt.timedelta(days=365)
    tf = (curr - prev).days / 365.25
    company[curr] = company[prev]
    bank[curr] = bank[prev]

    mv, gv, prem, cfs, _ = process_unit_linked_period(
        current_date=curr,
        previous_date=prev,
        portfolio_return=0.04,
        time=tf,
        mv_df=mv,
        gv_df=gv,
        premium_df=prem,
        policies=ul_policies,
        fund=ul_fund,
        society=society,
        random_seed=settings.random_seed,
        proj_period=k - 1,
    )
    bank[curr] += cfs["gross_premium"] - cfs["death"] - cfs["surrender"]
    company[curr] += cfs["entry_fee"] + cfs["admin_fee"]

    history.append({
        "date": curr,
        "UL reserve": ul_ptf.total_reserve(mv, curr),
        "Company account": float(company.loc[0, curr]),
        "Bank account": float(bank.loc[0, curr]),
        "In force": cfs["in_force"],
        "Deaths": cfs["deaths"],
        "Lapses": cfs["lapses"],
        "Gross premium": cfs["gross_premium"],
    })
```

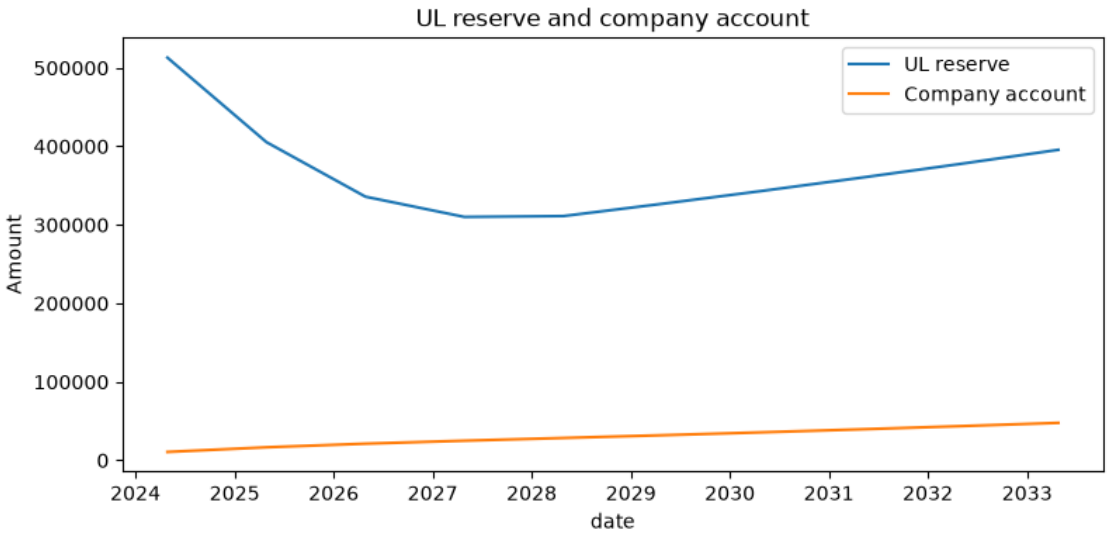
```
    prev = curr

history_df = pd.DataFrame(history).set_index("date")
display(history_df)

try:
    ax = history_df[["UL reserve", "Company account"]].plot(
        figsize=(9, 4), title="UL reserve and company account"
    )
    ax.set_ylabel("Amount")
except ImportError:
    print("Install matplotlib to plot this table (optional).")
```

```
              UL reserve  Company account   Bank account  In force  Deaths  \
date
2024-04-28  513035.715196     10656.393512 -427430.531293       8.0     1.0
2025-04-28  405408.090727     16584.850356 -549651.127525       7.0     0.0
2026-04-28  335916.965428     21196.058165 -630747.368643       6.0     0.0
2027-04-28  310104.782134     24964.112428 -666228.176292       5.0     0.0
2028-04-27  311193.118460     28453.485803 -674054.657877       4.0     0.0
2029-04-27  326852.704891     31954.132375 -667342.149612       4.0     0.0
2030-04-27  343104.758196     35622.852894 -660495.483984       4.0     0.0
2031-04-27  359969.394453     39465.911250 -653511.979699       4.0     0.0
2032-04-26  377467.376781     43489.782522 -646388.901876       4.0     0.0
2033-04-26  395620.135524     47701.159751 -639123.460975       4.0     0.0

            Lapses  Gross premium
date
2024-04-28     1.0   29013.506744
2025-04-28     1.0   15028.170606
2026-04-28     1.0   10022.702004
2027-04-28     1.0    6975.897051
2028-04-27     1.0    6713.460372
2029-04-27     0.0    6712.508265
2030-04-27     0.0    6846.665628
2031-04-27     0.0    6983.504285
2032-04-26     0.0    7123.077823
2033-04-26     0.0    7265.440901
```



1.11 Connection to the full ALM run

In `src/osem/main.py` with `liability_mode=unit_linked`:

1. Assets still evolve first (dividends, coupons, growth, bond reprice).
2. `Portfolio return` from the asset book capitalizes UL accounts.
3. UL cash flows update `bank_account` and `company_account`.
4. Proportional `trade()` invests / disinvests using **fund** cash only (`company_account` is excluded).

Switch the mode in `Input/Parameters.csv`:

```
liability_mode,unit_linked
random_seed,42
```

Then run the model as a module (`python -m osem.main`, or the `osem` console script after `pip install -e .`) and inspect `Output/Results.csv` for the UL summary columns, or open `Output/Report.html` for the charts.

The notebook finds the repository root itself, so it runs from any folder inside the repository.

1.12 Out of scope for this MVP

New business, lifecycle fund migration, full DCF reserving, company-account payouts, morbidity / retirement, and multi-fund asset buckets are not modelled yet.